Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Методические указания к домашнему заданию (часть 3) по дисциплине ОАД

Разработал:
Старший преподаватель
Масленников К.Ю.

Москва, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5
1 ФУНДАМЕНТАЛЬНЫЕ КОНЦЕПЦИИ KAFKA 6
1.1 Основные компоненты. Архитектурные элементы 6
1.2 Репликация и отказоустойчивость 7
1.2.1 Модель репликации 7
1.2.2 In-Sync Replicas (ISR) 8
2 АРХИТЕКТУРА KAFKA 9
2.1 Компоненты кластера Kafka 9
2.1.1 Broker 9
2.1.2 Zookeeper 10
2.2 Модель хранения данных 10
2.2.1 Структура хранения 10
2.2.2 Формат сообщения 11
3 ПРОИЗВОДИТЕЛИ (PRODUCERS) 12
3.1 Архитектура Producer 12
3.1.1 Компоненты Producer 12
3.2 Процесс отправки сообщений 13
3.2.1 Алгоритм отправки 13
3.2.2 Батчинг и компрессия 13
3.3 Гарантии доставки 14
3.3.1 Уровни подтверждения (Acks) 14
3.3.2 Идемпотентность и транзакции 15
4 ПОТРЕБИТЕЛИ (CONSUMERS) 16
4.1 Архитектура Consumer 16
4.1.1 Consumer Groups 16
4.1.2 Rebalancing 16
4.2 Стратегии распределения 17
4.2.1 Partition Assignment Strategies 17
4.3 Управление смещениями (Offsets) 18
4.3.1 Типы смещений 18
4.3.2 Политики сброса смещений 18
5 ПОТОКОВАЯ ОБРАБОТКА В KAFKA 19
5.1 Kafka Streams 19

5.1.1 Архитектура Kafka Streams 19
5.1.2 Абстракции потоковой обработки 20
5.2 Оконная обработка 20
5.2.1 Типы окон 20
5.2.2 Временные модели 21
6 УПРАВЛЕНИЕ И МОНИТОРИНГ 22
6.1 Администрирование кластера 22
6.1.1 Основные инструменты 22
6.1.2 Резюме 28
6.2 Мониторинг и метрики 29
6.2.1 Ключевые метрики 29
7 БЕЗОПАСНОСТЬ В KAFKA 30
7.1 Механизмы аутентификации 30
7.1.1 Поддерживаемые протоколы 30
7.2 Авторизация и шифрование 30
7.2.1 Модель авторизации 30
7.2.2 Шифрование данных 31
8 ОПТИМИЗАЦИЯ ПРОИЗВОДИТЕЛЬНОСТИ 32
8.1 Тюнинг конфигураций 32
8.1.1 Оптимизация Producer 32
8.1.2 Оптимизация Consumer 33
8.2 Проектирование топиков и партиций 34
8.2.1 Выбор количества партиций 34
8.2.2 Рекомендации по проектированию 34
9 ПОЛНАЯ УСТАНОВКА И НАСТРОЙКА КОМПОНЕНТОВ 35
9.1 Установка Apache Kafka 35
9.1.1 Вариант 1: Ручная установка (Linux) 35
9.1.2 Вариант 2: Установка через Docker 36
9.1.3 Проверка установки 37
9.2 Установка Apache Spark 38
9.2.1 Установка Spark 38
9.3 Установка ClickHouse 39
9.3.1 Установка через Docker 39
9.3.2 Проверка установки 39

9.4 Установка Grafana....40
9.5 Установка Python-библиотек....40
10 ЗАДАНИЕ....41
10.1 Часть 1. Основы Kafka....41
10.2 Часть 2. Потоковая обработка....42
10.3 Часть 3. Фрод-детекция....43
10.4 Часть 4. Рекомендательная система....44
10.5 Часть 5. Комплексный ETL-пайплайн....45
11 ПОРЯДОК ВЫПОЛНЕНИЯ....46
11.1 Общий алгоритм выполнения....46
12 ДОПОЛНИТЕЛЬНЫЕ МАТЕРИАЛЫ....47
12.1 Полезные команды Kafka....47
12.2 Рекомендуемые инструменты....48

ВВЕДЕНИЕ

Apache Kafka — это распределённая потоковая платформа, разработанная для обработки больших объёмов данных в реальном времени. Изначально созданная в LinkedIn, Kafka стала открытым проектом Apache и широко используется в современных системах для построения масштабируемых и отказоустойчивых потоковых приложений.

Kafka Apache — эффективный инструмент для организации работы серверных проектов любого уровня. Благодаря гибкости, масштабируемости и отказоустойчивости используется в различных направлениях IT-индустрии, от сервисов потоковых видео до аналитики Big Data.

1. Для связи микросервисов. Kafka — связующее звено между отдельными функциональными модулями большой системы. Например, с её помощью можно подписать микросервис на другие компоненты для регулярного получения обновлений.

2. Потоковая передача данных. Высокая пропускная способность системы позволяет поддерживать непрерывные потоки информации. За счёт грамотной маршрутизации «Кафка» не только надёжно передаёт данные, но и позволяет производить с ними различные операции.

3. Ведение журнала событий. Kafka сохраняет данные в строго организованную структуру, в которой всегда можно отследить, когда произошло то или иное событие. Информация хранится в течение заданного промежутка времени, что можно использовать для разгрузки базы данных или медленно работающих систем логирования.

1 ФУНДАМЕНТАЛЬНЫЕ КОНЦЕПЦИИ KAFKA

1.1 Основные компоненты. Архитектурные элементы

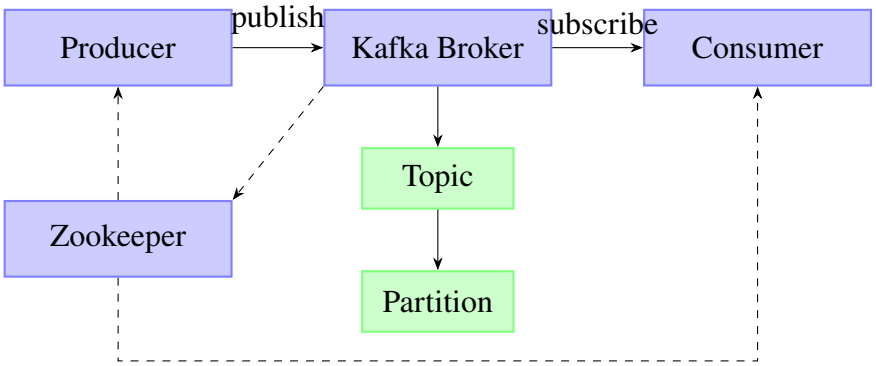


Рисунок 1 – Высокоуровневая архитектура Apache Kafka

Топик (Topics) — это категория или feed name, к которой публикуются сообщения. Каждый топик в Kafka является логическим каналом для передачи данных.

$$\text{Topic} = \{\text{Partition}_0, \text{Partition}_1, \ldots, \text{Partition}_N\}. \tag{1}$$

Свойства топиков

– **Идентификация**: Уникальное имя в рамках кластера.
– **Партиционирование**: Данные распределяются по партициям.
– **Репликация**: Обеспечение отказоустойчивости.
– **Хранение**: Сообщения хранятся настраиваемый период времени.

Партиции (Partitions)

Партиция — это упорядоченная, неизменяемая последовательность сообщений, которая постоянно добавляется в commit log.

Структура партиции

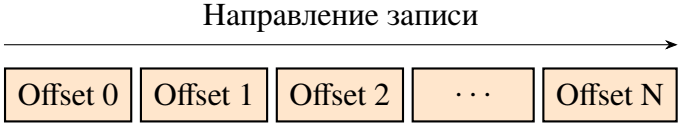


Рисунок 2 – Структура партиции как commit log

Ключевые характеристики партиций

– **Упорядоченность**: Сообщения в партиции сохраняют порядок.
– **Неизменяемость**: Данные только добавляются, не изменяются.
– **Персистентность**: Сообщения сохраняются на диск.
– **Сегментирование**: Лог разбивается на сегменты для управления.

1.2 Репликация и отказоустойчивость

1.2.1 Модель репликации

Kafka использует модель лидер-фолловер для репликации данных:

$$\text{Replication Factor} = N, \qquad \text{где } N \geq 1. \tag{2}$$

Алгоритм 1 Процесс репликации в Kafka

```
1:  procedure ReplicateMessage(message, partition)
2:      leader ← getLeader(partition)
3:      ISR ← getInSyncReplicas(partition)
4:      writeResult ← writeToLeader(leader, message)
5:      if writeResult.success then
6:          for all follower ∈ ISR do
7:              startAsyncReplication(follower, message)
8:          end for
9:      end if
10:     waitForReplication(ISR, min.insync.replicas)
11:     commitMessage(message)
12: end procedure
```

1.2.2 In-Sync Replicas (ISR)

ISR — это набор реплик, которые полностью синхронизированы с лидером:

$$ISR \subseteq \{\text{All Replicas}\}, \qquad |ISR| \geq \text{min.insync.replicas}. \tag{3}$$

2 АРХИТЕКТУРА KAFKA

2.1 Компоненты кластера Kafka

2.1.1 Broker

Брокер — это сервер Kafka, который хранит данные и обрабатывает запросы.

Таблица 1 – Компоненты Kafka Broker

Компонент	**Функциональность**
Socket Server	Обработка сетевых соединений
Request Handler	Обработка запросов производителей/потребителей
Storage Layer	Управление дисковым хранилищем
Replication Manager	Управление репликацией данных
Offset Manager	Управление смещениями потребителей

2.1.2 Zookeeper

Zookeeper выполняет координационные задачи в кластере Kafka:

– **Метаданные**: Хранение информации о брокерах, топиках, партициях.
– **Контроллер**: Выбор контроллера кластера.
– **Конфигурация**: Хранение конфигураций топиков.
– **Кворум**: Координация распределённых операций.

2.2 Модель хранения данных

2.2.1 Структура хранения

Kafka использует log-structured storage с сегментированием:

$$\text{Partition} = \{\text{Segment}_0, \text{Segment}_1, \ldots, \text{Segment}_M\}. \tag{4}$$

Каждый сегмент состоит из:

– **.log file**: Файл с сообщениями.
– **.index file**: Индекс для быстрого поиска.
– **.timeindex file**: Временной индекс.

2.2.2 Формат сообщения

Таблица 2 – Формат сообщения в Kafka

Поле	**Размер**	**Описание**
Offset	8 bytes	Уникальный идентификатор
Message Size	4 bytes	Размер сообщения
CRC	4 bytes	Контрольная сумма
Magic Byte	1 byte	Версия формата
Attributes	1 byte	Атрибуты сообщения
Key Length	4 bytes	Длина ключа
Key	Variable	Ключ сообщения
Value Length	4 bytes	Длина значения
Value	Variable	Тело сообщения

3 ПРОИЗВОДИТЕЛИ (PRODUCERS)

3.1 Архитектура Producer

3.1.1 Компоненты Producer

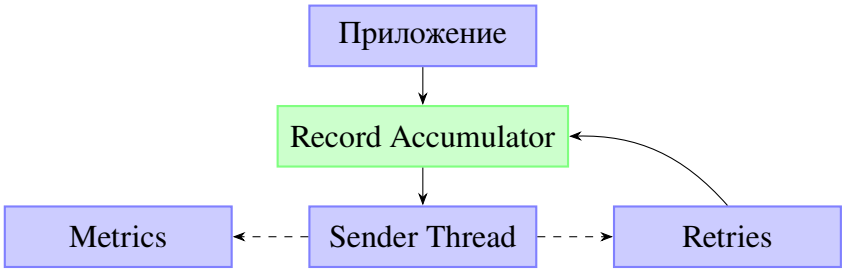


Рисунок 3 – Архитектура Kafka Producer

3.2 Процесс отправки сообщений

3.2.1 Алгоритм отправки

Алгоритм 2 Алгоритм отправки сообщения Producer

```
1: procedure SendMessage(topic, key, value)
2:     partition ← calculatePartition(topic, key)
3:     batch ← getOrCreateBatch(partition)
4:     addToBatch(batch, key, value)
5:     if batch.isFull() ∨ batch.isExpired() then
6:         sender.wakeup
7:     end if
8:     return Future[RecordMetadata]
9: end procedure
```

3.2.2 Батчинг и компрессия

- **Батчинг**: Группировка сообщений для уменьшения сетевых запросов.
- **Компрессия**: Сжатие данных для уменьшения трафика.
- **Тайминги**: Настройка задержек для оптимизации батчинга.

3.3 Гарантии доставки

3.3.1 Уровни подтверждения (Acks)

Таблица 3 – Уровни подтверждения доставки

Уровень	**Значение**	**Гарантии**
acks=0	No acknowledgment	Максимальная производительность, возможна потеря данных
acks=1	Leader only	Лидер подтверждает запись, возможна потеря при сбое
acks=all	All replicas	Все реплики подтверждают, максимальная надёжность

3.3.2 Идемпотентность и транзакции

- **Идемпотентность**: Гарантия однократной доставки.
- **Транзакции**: Атомарность между несколькими партициями.
- **Sequential IDs**: Уникальные идентификаторы для отслеживания.

4 ПОТРЕБИТЕЛИ (CONSUMERS)

4.1 Архитектура Consumer

4.1.1 Consumer Groups

Consumer Group — это набор потребителей, совместно обрабатывающих данные топика:

$$\forall p \in \text{Partitions} \quad \exists!\, c \in \text{Consumers}\colon \text{assigned}(p, c). \tag{5}$$

4.1.2 Rebalancing

Процесс перераспределения партиций между потребителями:

Алгоритм 3 Процесс Rebalancing в Consumer Group

```
1:  procedure Rebalance(group, members, partitions)
2:      coordinator ← findCoordinator(group)
3:      strategy ← selectAssignmentStrategy
4:      assignment ← strategy.assign(members, partitions)
5:      for all member ∈ members do
6:          revokeCurrentAssignment(member)
7:          sendNewAssignment(member, assignment[member])
8:      end for
9:      syncGroupState(assignment)
10: end procedure
```

4.2 Стратегии распределения

4.2.1 Partition Assignment Strategies

Таблица 4 – Стратегии распределения партиций

Стратегия	**Принцип работы**	**Преимущества**
RangeAssignor	Распределение по диапазонам партиций	Простота реализации
RoundRobinAssignor	Циклическое распределение	Равномерная нагрузка
StickyAssignor	Минимизация изменений при rebalance	Стабильность назначений
CooperativeStickyAssignor	Инкрементальный rebalance	Уменьшение downtime

4.3 Управление смещениями (Offsets)

4.3.1 Типы смещений

- **Current Offset**: Текущая позиция потребителя.
- **Committed Offset**: Подтверждённая позиция.
- **Log End Offset (LEO)**: Последнее сообщение в партиции.
- **High Watermark**: Последнее подтверждённое сообщение.

4.3.2 Политики сброса смещений

Таблица 5 – Политики сброса смещений потребителя

Политика	**Поведение**
earliest	Начать с самого старого сообщения
latest	Начать с самого нового сообщения
none	Выбросить исключение, если смещение не найдено
specific	Начать с указанного смещения

5 ПОТОКОВАЯ ОБРАБОТКА В KAFKA

Потоковая обработка в Kafka — это непрерывная обработка данных в реальном времени с использованием Kafka Streams, библиотеки для создания приложений, которые получают, трансформируют и отправляют данные из Kafka. Архитектура потоковой обработки строится на основе графа: узлы-процессоры выполняют операции над данными, а рёбра-потоки передают их между узлами.

5.1 Kafka Streams

5.1.1 Архитектура Kafka Streams

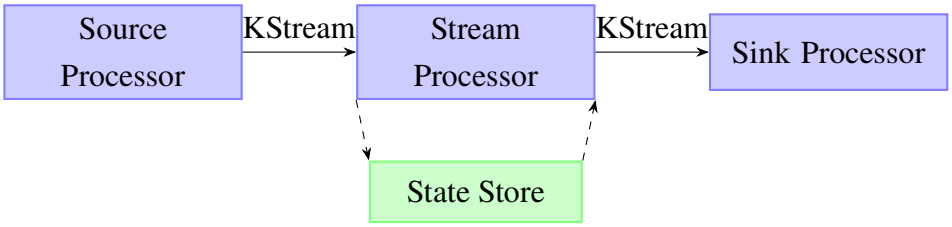


Рисунок 4 – Архитектура Kafka Streams приложения

5.1.2 Абстракции потоковой обработки

– **KStream**: Поток всех записей в топике.
– **KTable**: Changelog stream, представляющий таблицу.
– **GlobalKTable**: Полностью реплицируемая таблица.
– **KTables**: Поток, где каждая запись — это новая версия.

5.2 Оконная обработка

5.2.1 Типы окон

Таблица 6 – Типы окон в потоковой обработке

Тип окна	**Описание**	**Использование**
Tumbling Window	Непересекающиеся фиксированные окна	Агрегация по фиксированным интервалам
Hopping Window	Пересекающиеся окна с фиксированным шагом	Скользящие агрегаты
Sliding Window	Окна, определяемые временем события	Сессии с временными границами
Session Window	Окна на основе периодов неактивности	Анализ пользовательских сессий

5.2.2 Временные модели

- **Event Time**: Время создания события (наиболее точное).
- **Processing Time**: Время обработки события (простое).
- **Ingestion Time**: Время поступления в Kafka (компромисс).

6 УПРАВЛЕНИЕ И МОНИТОРИНГ

6.1 Администрирование кластера

6.1.1 Основные инструменты

– **kafka-topics**: Управление топиками.
– **kafka-configs**: Управление конфигурациями.
– **kafka-consumer-groups**: Управление consumer groups.
– **kafka-acls**: Управление доступом.
– **kafka-reassign-partitions**: Перераспределение партиций.

Это основные утилиты командной строки для администрирования Apache Kafka. Давайте подробно разберём каждую из них, их ключевые функции и примеры использования.

6.1.1.1 kafka-topics: Управление топиками

Эта утилита используется для всех операций с топиками: создание, удаление, просмотр, изменение.

Ключевые операции:

– **Создание топика:**

```
kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create \
  --topic my-topic \
  --partitions 3 \
  --replication-factor 1
```

– –partitions: Количество партиций для параллельной обработки.

– –replication-factor: Фактор репликации для отказоустойчивости.

– **Просмотр списка топиков:**

```
kafka-topics.sh \
  --bootstrap-server localhost:9092 \
```

```
  --list
```

– **Полное описание топика:**

```
kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --describe \
  --topic my-topic
```

Показывает партиции, их лидеры, реплики и статус ISR (in-sync replicas).

– **Удаление топика:**

```
kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --delete \
  --topic my-topic
```

Важно: Убедитесь, что в конфигурации брокера delete.topic.enable=true.

– **Изменение количества партиций:**

```
kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --alter \
  --topic my-topic \
  --partitions 5
```

Примечание: Можно только увеличивать, уменьшать нельзя.

6.1.1.2 kafka-configs: Управление конфигурациями

Позволяет просматривать и изменять конфигурации топиков, брокеров и клиентов «на лету».

Ключевые операции:

– **Добавить конфигурацию для топика:**

```
kafka-configs.sh \
  --bootstrap-server localhost:9092 \
  --entity-type topics \
  --entity-name my-topic \
  --alter \
  --add-config retention.ms=3600000
```

Устанавливает время хранения сообщений в топике my-topic до 1 часа.

– **Просмотреть конфигурацию топика:**

```
kafka-configs.sh \
  --bootstrap-server localhost:9092 \
  --entity-type topics \
  --entity-name my-topic \
  --describe
```

– **Удалить конфигурацию:**

```
kafka-configs.sh \
  --bootstrap-server localhost:9092 \
  --entity-type topics \
  --entity-name my-topic \
  --alter \
  --delete-config retention.ms
```

6.1.1.3 kafka-consumer-groups: Управление consumer groups

Инструмент для мониторинга и управления потребительскими группами.

Ключевые операции:

– **Список всех consumer groups:**

```
kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --list
```

– **Описание группы (самая важная команда):**

```
kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --group my-group \
  --describe
```

Показывает:

– CURRENT-OFFSET: Текущий оффсет консьюмера.

– LOG-END-OFFSET: Последний оффсет в партиции.

– LAG: Отставание (LOG-END-OFFSET – CURRENT-OFFSET).

– Какой консьюмер (HOST) читает какую партицию.

– **Сброс оффсетов:**

```
# Сброс на начало (earliest)
kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --group my-group \
  --reset-offsets \
  --to-earliest \
  --execute \
  --topic my-topic

# Сброс на конец (latest)
kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --group my-group \
  --reset-offsets \
  --to-latest \
  --execute \
  --topic my-topic

# Сброс на определенный оффсет
kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --group my-group \
  --reset-offsets \
  --to-offset 100 \
  --execute \
  --topic my-topic
```

Важно: Перед сбросом убедитесь, что все консьюмеры в группе остановлены.

6.1.1.4 kafka-acls: Управление доступом

Используется для настройки авторизации и контроля доступа (ACL — Access Control List) в Kafka.

Ключевые операции:

- **Добавить ACL, разрешающий чтение топика:**

```
kafka-acls.sh \
  --bootstrap-server localhost:9092 \
  --add \
  --allow-principal User:consumer1 \
```

```
  --operation Read \
  --topic my-topic \
  --command-config /path/to/admin.properties
```

– Добавить ACL, разрешающий запись в топик:

```
kafka-acls.sh \
  --bootstrap-server localhost:9092 \
  --add \
  --allow-principal User:producer1 \
  --operation Write \
  --topic my-topic \
  --command-config /path/to/admin.properties
```

– Добавить ACL для всей consumer group:

```
kafka-acls.sh \
  --bootstrap-server localhost:9092 \
  --add \
  --allow-principal User:consumer1 \
  --operation Read \
  --group my-group \
  --command-config /path/to/admin.properties
```

– Просмотреть все ACL для топика:

```
kafka-acls.sh \
  --bootstrap-server localhost:9092 \
  --list \
  --topic my-topic \
  --command-config /path/to/admin.properties
```

– Удалить ACL:

```
kafka-acls.sh \
  --bootstrap-server localhost:9092 \
  --remove \
  --allow-principal User:consumer1 \
  --operation Read \
  --topic my-topic \
  --command-config /path/to/admin.properties
```

Примечание: Для работы с ACL требуется настройка безопасности Kafka (SASL/SSL). admin.properties *содержит учётные данные администратора.*

6.1.1.5 kafka-reassign-partitions: Перераспределение партиций

Мощный инструмент для перераспределения реплик партиций между брокерами. Используется для:

- Балансировки нагрузки.
- Добавления новых брокеров в кластер.
- Вывода брокеров из кластера (декоммишн).

Процесс состоит из трёх этапов:

1. **Генерация плана перераспределения:**

Создаётся JSON-файл (reassignment-plan.json), в котором указывается, какие топики и партиции нужно переместить.

```
kafka-reassign-partitions.sh \
  --bootstrap-server localhost:9092 \
  --generate \
  --topics-to-move-json-file topics-to-move.json \
  --broker-list 1,2,3
```

2. **Выполнение плана:**

Запускается процесс перераспределения.

```
kafka-reassign-partitions.sh \
  --bootstrap-server localhost:9092 \
  --execute \
  --reassignment-json-file reassignment-plan.json
```

3. **Проверка статуса:**

Мониторинг хода выполнения операции.

```
kafka-reassign-partitions.sh \
  --bootstrap-server localhost:9092 \
  --verify \
  --reassignment-json-file reassignment-plan.json
```

Важно: Эта операция создаёт высокую нагрузку на сеть и диски, поэтому её следует выполнять в периоды низкой нагрузки.

6.1.2 Резюме

Таблица 7 – Утилиты администрирования Kafka

Утилита	**Назначение**	**Ключевой Use-Case**
kafka-topics	Управление топиками	Создание, удаление, описание топиков
kafka-configs	Управление конфигурациями	Изменение политики хранения, сжатия и т. д.
kafka-consumer-groups	Управление группами	Мониторинг отставания (lag) и сброс оффсетов
kafka-acls	Управление доступом	Настройка прав для пользователей и приложений
kafka-reassign-partitions	Балансировка данных	Добавление/удаление брокеров, выравнивание нагрузки

Это обязательный набор инструментов для любого администратора Kafka, позволяющий полностью контролировать состояние и производительность кластера.

6.2 Мониторинг и метрики

6.2.1 Ключевые метрики

Таблица 8 – Ключевые метрики для мониторинга Kafka

Категория	**Метрика**	**Значимость**
Производительность	Messages per second	Пропускная способность
Задержки	Produce/Consume latency	Время обработки
Потребители	Consumer lag	Отставание потребителей
Брокеры	Under-replicated partitions	Проблемы репликации
Системные	CPU/Memory/Disk usage	Использование ресурсов

7 БЕЗОПАСНОСТЬ В KAFKA

7.1 Механизмы аутентификации

7.1.1 Поддерживаемые протоколы

- **SASL/PLAIN**: Простая аутентификация по логину/паролю.
- **SASL/SCRAM**: Более безопасная аутентификация.
- **SASL/GSSAPI**: Интеграция с Kerberos.
- **SASL/OAUTHBEARER**: Аутентификация через OAuth 2.0.
- **SSL Client Authentication**: Аутентификация через SSL-сертификаты.

7.2 Авторизация и шифрование

7.2.1 Модель авторизации

- **ACLs**: Access Control Lists для тонкой настройки прав.
- **RBAC**: Role-Based Access Control.
- **Resource Types**: Топики, группы, кластеры и т. д.
- **Operations**: CREATE, READ, WRITE, DELETE, DESCRIBE.

7.2.2 Шифрование данных

- **SSL/TLS**: Шифрование данных в движении.
- **Encryption at rest**: Шифрование данных на диске.
- **Field-level encryption**: Шифрование отдельных полей сообщений.

8 ОПТИМИЗАЦИЯ ПРОИЗВОДИТЕЛЬНОСТИ

8.1 Тюнинг конфигураций

8.1.1 Оптимизация Producer

Таблица 9 – Оптимизации для Producer

Параметр	**Рекомендация**	**Влияние**
batch.size	16–64 KB	Увеличение размера батча
linger.ms	10–100 ms	Увеличение времени батчинга
compression.type	snappy/lz4	Сжатие данных
buffer.memory	32–64 MB	Размер буфера отправки
acks	1/all	Баланс надёжности и производительности

8.1.2 Оптимизация Consumer

Таблица 10 – Оптимизации для Consumer

Параметр	**Рекомендация**	**Влияние**
fetch.min.bytes	1–10 KB	Минимальный размер fetch-запроса
fetch.max.wait.ms	100–500 ms	Максимальное время ожидания fetch
max.partition.fetch.bytes	1–10 MB	Максимальный размер на партицию
session.timeout.ms	10–30 seconds	Таймаут сессии потребителя
heartbeat.interval.ms	3–10 seconds	Интервал heartbeat

8.2 Проектирование топиков и партиций

8.2.1 Выбор количества партиций

$$N_{\text{partitions}} = \max\left(N_{\text{consumers}}, N_{\text{producers}}\right) \times R, \tag{6}$$

где R — коэффициент резервирования (обычно 1.5–2.0).

8.2.2 Рекомендации по проектированию

- **Размер сообщения**: Оптимально 1 KB–1 MB.
- **Ключи сообщений**: Для гарантии порядка обработки.
- **Время хранения**: Баланс между стоимостью и требованиями.
- **Репликация**: Минимум 3 реплики для production.

9 ПОЛНАЯ УСТАНОВКА И НАСТРОЙКА КОМПОНЕНТОВ

9.1 Установка Apache Kafka

9.1.1 Вариант 1: Ручная установка (Linux)

1. **Установка зависимостей:**

```
sudo apt update
sudo apt install openjdk-11-jdk wget curl gnupg -y
```

2. **Скачивание и установка Kafka:**

```
wget https://downloads.apache.org/kafka/3.4.0/kafka_2.13-3.4.0.tgz
tar -xzf kafka_2.13-3.4.0.tgz
cd kafka_2.13-3.4.0
```

3. **Настройка Zookeeper:**

```
# Редактирование config/zookeeper.properties
dataDir=/tmp/zookeeper
clientPort=2181
maxClientCnxns=100
```

4. **Настройка Kafka Broker:**

```
# Редактирование config/server.properties
broker.id=0
listeners=PLAINTEXT://:9092
advertised.listeners=PLAINTEXT://localhost:9092
log.dirs=/tmp/kafka-logs
num.partitions=3
offsets.topic.replication.factor=1
transaction.state.log.replication.factor=1
```

9.1.2 Вариант 2: Установка через Docker

1. **Установка Docker и Docker Compose:**

```
sudo apt install docker.io docker-compose -y
sudo usermod -aG docker $USER
```

2. **Создание** docker-compose.yml**:**

```
version: '3.8'
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:latest
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - "2181:2181"

  kafka:
    image: confluentinc/cp-kafka:latest
    depends_on:
      - zookeeper
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
    ports:
      - "9092:9092"

  kafka-ui:
    image: provectuslabs/kafka-ui:latest
    depends_on:
      - kafka
```

```
    environment:
      KAFKA_CLUSTERS_0_NAME: local
      KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:9092
      KAFKA_CLUSTERS_0_ZOOKEEPER: zookeeper:2181
    ports:
      - "8080:8080"
```

3. **Запуск сервисов:**

```
docker-compose up -d
```

9.1.3 Проверка установки

```
# Проверка работы Zookeeper
echo stat | nc localhost 2181

# Проверка работы Kafka
docker exec -it kafka-kafka-1 kafka-topics \
  --list \
  --bootstrap-server localhost:9092

# Создание тестового топика
kafka-topics --create \
  --topic test-topic \
  --partitions 3 \
  --replication-factor 1 \
  --bootstrap-server localhost:9092
```

9.2 Установка Apache Spark

9.2.1 Установка Spark

1. **Скачивание и установка:**

```
wget \
  https://downloads.apache.org/spark/spark-3.4.0/spark-3.4.0-bin-hadoop3.tgz
tar -xzf spark-3.4.0-bin-hadoop3.tgz
cd spark-3.4.0-bin-hadoop3
```

2. **Настройка окружения:**

```
echo 'export SPARK_HOME=/path/to/spark-3.4.0-bin-hadoop3' >> ~/.bashrc
echo 'export PATH=$SPARK_HOME/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

3. **Проверка установки:**

```
spark-submit --version
```

9.3 Установка ClickHouse

9.3.1 Установка через Docker

```
docker run -d \
  --name clickhouse \
  -p 8123:8123 -p 9000:9000 \
  -v /path/to/clickhouse/data:/var/lib/clickhouse \
  --ulimit nofile=262144:262144 \
  clickhouse/clickhouse-server
```

9.3.2 Проверка установки

```
-- Подключение к ClickHouse
docker exec -it clickhouse clickhouse-client

-- Создание тестовой базы данных
CREATE DATABASE kafka_labs;
USE kafka_labs;
```

9.4 Установка Grafana

```
docker run -d \
  --name grafana \
  -p 3000:3000 \
  -v /path/to/grafana/data:/var/lib/grafana \
  grafana/grafana
```

9.5 Установка Python-библиотек

```
pip install confluent-kafka kafka-python pyspark
pip install clickhouse-driver clickhouse-connect
pip install pandas numpy scikit-learn matplotlib
pip install jupyter notebook
```

10 ЗАДАНИЕ

10.1 Часть 1. Основы Kafka

Тема: «Система сбора и обработки логов веб-приложения».

Цель: Освоить базовые принципы работы с Apache Kafka, создать систему сбора логов.

Задачи:

1. Настроить Kafka кластер.
2. Создать топики для логов разного уровня.
3. Реализовать Producer для генерации логов.
4. Реализовать Consumer для обработки логов.
5. Настроить фильтрацию логов по уровню важности.

Требования:

– Топики: web-logs, error-logs, debug-logs.

– Поддержка уровней логирования: INFO, WARN, ERROR, DEBUG.

– Фильтрация ERROR логов в отдельный топик.

– Сохранение статистики в файл.

Вариант 1: Система логирования микросервисов.

Вариант 2: Сбор метрик производительности.

Вариант 3: Мониторинг сетевой активности.

10.2 Часть 2. Потоковая обработка

Тема: «Система мониторинга IoT-устройств».

Цель: Реализовать систему сбора и обработки данных с IoT-устройств.

Задачи:

1. Создать топик для данных датчиков.
2. Реализовать генератор данных IoT-устройств.
3. Настроить потоковую обработку в Spark Streaming.
4. Реализовать обнаружение аномалий.
5. Визуализировать данные в Grafana.

Требования:

– Поддержка 100+ виртуальных устройств.

– Обнаружение аномальных показаний.

– Агрегация данных по временным окнам.

– Визуализация в реальном времени.

Вариант 1: Умный город (датчики окружающей среды).

Вариант 2: Промышленный IoT (мониторинг оборудования).

Вариант 3: Умный дом (системы управления).

10.3 Часть 3. Фрод-детекция

Тема: «Система обнаружения мошеннических операций».

Цель: Создать систему реального времени для обнаружения подозрительных транзакций.

Задачи:

1. Спроектировать схему данных для транзакций.
2. Реализовать генератор банковских транзакций.
3. Настроить правила обнаружения мошенничества.
4. Интегрировать ML модель для классификации.
5. Реализовать систему оповещений.

Требования:

– Обнаружение 5+ типов мошеннических паттернов.
– Интеграция простой ML модели.
– Задержка обработки < 2 секунд.
– Система приоритетных оповещений.

Вариант 1: Банковское мошенничество.

Вариант 2: E-commerce мошенничество.

Вариант 3: Страховое мошенничество.

10.4 Часть 4. Рекомендательная система

Тема: «Платформа рекомендаций в реальном времени».

Цель: Построить систему рекомендаций на основе пользовательского поведения.

Задачи:

1. Создать топики для пользовательских событий.
2. Реализовать сбор пользовательских взаимодействий.
3. Настроить обработку событий в реальном времени.
4. Реализовать алгоритмы рекомендаций.
5. Создать API для выдачи рекомендаций.

Требования:

– Поддержка collaborative filtering.

– Учёт контекстной информации.

– Генерация рекомендаций < 100 мс.

– A/B тестирование алгоритмов.

Вариант 1: Рекомендации контента (видео, музыка).

Вариант 2: Рекомендации товаров (e-commerce).

Вариант 3: Рекомендации новостей.

10.5 Часть 5. Комплексный ETL-пайплайн

Тема: «Построение end-to-end пайплаина обработки данных».

Цель: Интегрировать все изученные компоненты в единую систему.

Задачи:

1. Настроить Kafka Connect для интеграции с внешними системами.
2. Реализовать сложную потоковую обработку.
3. Настроить мониторинг и алертинг.
4. Оптимизировать производительность системы.
5. Реализовать обработку ошибок.

Требования:

– Интеграция 3+ источников данных.
– Обработка 1000+ сообщений в секунду.
– Мониторинг ключевых метрик.
– Отказоустойчивость при сбоях.

1. **Вариант 1:** Платформа аналитики e-commerce.
2. **Вариант 2:** Система мониторинга IT-инфраструктуры.
3. **Вариант 3:** Платформа обработки финансовых данных.

11 ПОРЯДОК ВЫПОЛНЕНИЯ

11.1 Общий алгоритм выполнения

1. **Изучение теоретического материала.**
2. **Установка и настройка необходимых компонентов.**
3. **Проектирование архитектуры решения.**
4. **Реализация отдельных компонентов системы.**
5. **Интеграция компонентов и тестирование.**
6. **Оптимизация и документирование.**
7. **Подготовка отчёта и демонстрация.**

12 ДОПОЛНИТЕЛЬНЫЕ МАТЕРИАЛЫ

12.1 Полезные команды Kafka

```
# Просмотр списка топиков
kafka-topics --list \
  --bootstrap-server localhost:9092

# Создание топика
kafka-topics --create \
  --topic my-topic \
  --partitions 3 \
  --replication-factor 1 \
  --bootstrap-server localhost:9092

# Просмотр информации о топике
kafka-topics --describe \
  --topic my-topic \
  --bootstrap-server localhost:9092

# Отправка сообщений
kafka-console-producer \
  --topic my-topic \
  --bootstrap-server localhost:9092

# Чтение сообщений
kafka-console-consumer \
  --topic my-topic \
  --from-beginning \
  --bootstrap-server localhost:9092

# Просмотр смещений
kafka-consumer-groups --list \
  --bootstrap-server localhost:9092
```

```
kafka-consumer-groups --describe \
  --group my-group \
  --bootstrap-server localhost:9092
```

12.2 Рекомендуемые инструменты

- **Kafka UI**: Веб-интерфейс для управления Kafka.
- **Kafka Tool**: GUI приложение для работы с Kafka.
- **Kafka Manager**: Веб-инструмент для управления кластером.
- **Jupyter Notebook**: Для экспериментов с кодом.
- **Postman**: Для тестирования API.